

The Answer Is Not a New HTAP System: Fresher Analytics Through Instance-Optimized Data Orchestration

Geoffrey X. Yu
Massachusetts Institute of Technology
geoffxy@mit.edu

Over the past two decades, the database community has worked on building specialized systems for transactional workloads (OLTP systems) [2] and analytical workloads (OLAP systems) [3]. This specialization makes sense because transactional and analytical workloads benefit from opposing kinds of optimizations (e.g., a row versus columnar physical data layout) and typically have very different access patterns (e.g., a few point accesses versus table scans). As a result, a common data architecture is to deploy both a transactional database management system (DBMS) and an analytical DBMS (e.g., a data warehouse) in order to get the best possible performance for both kinds of workloads. Benefits aside, a key disadvantage of this multi-system architecture is the latency to insight: the data warehouse does not always have the latest data. Practitioners typically employ a data pipeline that periodically (e.g., nightly or every few hours) copies the latest writes from the transactional DBMS into the warehouse. While this periodic copy was suitable in the past, companies today increasingly want to make decisions using the latest fresh data. Consequently, there has recently been a renewed interest in unifying OLTP and OLAP systems as one hybrid transactional analytical (HTAP) DBMS that supports both transactional and analytical workloads. For example, companies have been (i) adding analytical support to traditional OLTP DBMSes (e.g., Google’s AlloyDB, Oracle’s MySQL Heatwave), and (ii) building entirely new HTAP systems (e.g., TiDB and SingleStore).

While these systems are promising, we believe that addressing this latency-to-insight problem does not necessitate building a new DBMS that replaces existing OLTP and OLAP systems. Instead, we think that it is possible to significantly accelerate access to fresh data if we take advantage of and automatically adapt to *workload-specific characteristics* (such as read skew and write skew)—what we refer to as “instance-optimized data orchestration.” We outline two of our ideas related to this theme: (i) leveraging compensation queries across database systems with strategic pre-materialization, and (ii) partitioning data to accelerate both reads and ingestion.

Compensation Queries and Strategic Pre-Materialization Across Existing DBMSes. When running a query on a data warehouse, the freshness you achieve depends on how frequently the latest writes (from the transactional DBMS) are copied over.¹ Instead of waiting for these copies, we propose leveraging the transactional DBMS to get access to the latest writes. For a given analytical query, the key idea is to treat the results from the OLAP system as a materialized view V over the query’s base tables. If we have the inserts and deletes (the “deltas”) made to the base tables, then we can apply incremental view maintenance delta propagation to generate *compensation queries* (issued to the transactional DBMS) that convert these base table deltas to deltas on V [1]. This approach is advantageous when computing the deltas to V (on the transactional DBMS) and then merging them into V is faster than replicating the base table deltas onto the OLAP DBMS. The key challenge in making this approach efficient is incrementally maintaining the result of a join. For example, if you have inserts to B and want to compute the inserts to $A \bowtie B$, you need to join the inserts with A , which itself can be a complex relational algebra expression that is expensive to compute. Ideally, we would pre-materialize A , but this can be problematic if A is large. Our key observation is that we only need to materialize the subset of A that could possibly join with the writes to B . If the writes to B are skewed (common in practice), then we may only need to materialize a small subset of A . Realizing this idea requires developing (i) a method to detect write skew and to generate beneficial intermediate result subsets to materialize for a given storage budget, and (ii) a query planner that generates compensation queries leveraging these pre-materialized results. Designing such a method and planner is non-trivial as they depend on (i) how the underlying data is read and written, and (ii) the performance characteristics of each DBMS.

Accelerating Data Ingestion Through Read and Write Optimized Partitioning Schemes. A common operation when updating a data warehouse is a MERGE, which merges inserts and updates from a staging table (the data being ingested) into the underlying base table. This operation is expensive because it requires a join: the warehouse must scan its entire base table and look for matches between the base table and the staging table. Matches represent updates; unmatched rows are inserts. One way to accelerate this merge is through *workload-aware partitioning*: if you can be sure that none of the rows in a partition will match any data in the staging table, then you can skip the partition entirely. Data warehouses today will usually partition tables to accelerate *reads*: if you know that your queries filter by a few specific columns, then it is advantageous to partition based on them [4]. But a read-optimized partitioning can be a poor choice for data ingestion if the rows being updated are spread throughout many partitions (i.e., not skewed along the partitioned dimensions). By instead considering which regions of the table are updated along with how the tables are queried, we believe that we can choose an effective partitioning that maximizes the amount of data skipped when executing both read queries *and* data ingestion.

References

- [1] Timothy Griffin and Leonid Libkin. Incremental maintenance of views with duplicates. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*, SIGMOD ’95, page 328–339, New York, NY, USA, 1995. Association for Computing Machinery.
- [2] Robert Kallman, Hideaki Kimura, Jonathan Natkins, Andrew Pavlo, Alexander Rasin, Stanley Zdonik, Evan PC Jones, Samuel Madden, Michael Stonebraker, Yang Zhang, et al. H-Store: A High-Performance, Distributed Main Memory Transaction Processing System. *Proceedings of the VLDB Endowment*, 1(2):1496–1499, 2008.
- [3] Mike Stonebraker, Daniel J Abadi, Adam Batkin, Xuedong Chen, Mitch Cherniack, Miguel Ferreira, Edmond Lau, Amerson Lin, Sam Madden, Elizabeth O’Neil, et al. C-Store: A Column-Oriented DBMS. In *Proceedings of the 31st VLDB Conference (VLDB’05)*, 2005.
- [4] Zongheng Yang, Badrish Chandramouli, Chi Wang, Johannes Gehrke, Yinan Li, Umar Farooq Minhas, Per-Åke Larson, Donald Kossmann, and Rajeev Acharya. Qd-Tree: Learning Data Layouts for Big Data Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD’20)*, page 193–208, 2020.

¹ Here we are assuming a two system architecture (one OLTP and one OLAP DBMS) that have the same schema and where writes originate on the OLTP system.

Biography

Geoffrey is a fourth year PhD student in the Data Systems Group at MIT, advised by Tim Kraska. Geoffrey currently works on BRAD: a new system that automatically designs and operates multi-engine cloud data infrastructures—powered by learned cost-based optimization. He is also broadly interested in cloud-native database management systems, systems-oriented database problems, and large-scale systems. Before starting his PhD, Geoffrey completed a master's degree in Computer Science at the University of Toronto where he worked on GPU performance prediction models to help deep learning researchers and practitioners select the best GPU for their training workloads. Geoffrey received his Bachelor in Software Engineering from the University of Waterloo. Geoffrey is a recipient of the NSERC Canada Graduate Scholarship – Doctoral (2020) and the Snap Research Scholarship (2019).

Website: www.geoffreyyu.com